

bok25

基
礎
班

、**考研数据結構**



排序



排序

定义：关键字的非递减或非递增顺序对一组记录重新进行排列的操作



排序

定义：关键字的非递减或非递增顺序对一组记录重新进行排列的操作

概念：

稳定性：有两个关键字相同的元素，经过排序后，他们的相对顺序仍然不变，即为算法是稳定的

待排序序列

1	3	2	2	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

稳定排序后结果

待排序序列

1	3	2	2	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

不稳定排序后结果



排序

定义：关键字的非递减或非递增顺序对一组记录重新进行排列的操作

概念：

稳定性：有两个关键字相同的元素，经过排序后，他们的相对顺序仍然不变，即为算法是稳定的

待排序序列	1	3	2	2	4	5	6	7	8	9
-------	---	---	---	---	---	---	---	---	---	---

稳定排序后结果	1	2	2	3	4	5	6	7	8	9
---------	---	---	---	---	---	---	---	---	---	---

待排序序列	1	3	2	2	4	5	6	7	8	9
-------	---	---	---	---	---	---	---	---	---	---

不稳定排序后结果	1	2	2	3	4	5	6	7	8	9
----------	---	---	---	---	---	---	---	---	---	---



排序

定义：关键字的非递减或非递增顺序对一组记录重新进行排列的操作

概念：

稳定性：有两个关键字相同的元素，经过排序后，他们的相对顺序仍然不变，即为算法是稳定的

稳定排序相对顺序不变

待排序序列

1	3	2	2	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

稳定排序后结果

1	2	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

待排序序列

1	3	2	2	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

不稳定排序后结果

1	2	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---



排序

定义：关键字的非递减或非递增顺序对一组记录重新进行排列的操作

概念：

稳定性：有两个关键字相同的元素，经过排序后，他们的相对顺序仍然不变，即为算法是稳定的

待排序序列	1	3	2	2	4	5	6	7	8	9
稳定排序后结果	1	2	2	3	4	5	6	7	8	9

不稳定排序相对顺序发生改变！

红蓝变成了蓝红

待排序序列	1	3	2	2	4	5	6	7	8	9
不稳定排序后结果	1	2	2	3	4	5	6	7	8	9



排序

定义：关键字的非递减或非递增顺序对一组记录重新进行排列的操作

概念：

稳定性：有两个关键字相同的元素，经过排序后，他们的相对顺序仍然不变，即为算法是稳定的

内部排序：排序期间元素全部存放在内存中的排序

外部排序：内存一次不能容纳全部记录，在排序过程中也需要对外存进行访问的排序过程



内部排序

插入排序



插入排序

直接插入排序



插入排序

直接插入排序

核心思想

-	49	38	65	97	76	13	27	49
---	----	----	----	----	----	----	----	----

未排序序列



插入排序

直接插入排序

核心思想

1. 分割为已排序和未排序两组
2. 每次把未排序序列的第一个插入到已排序序列的合适位置

已有序序列

未排序序列

49	38	65	97	76	13	27	49
----	----	----	----	----	----	----	----



插入排序

直接插入排序

核心思想

1. 分割为已排序和未排序两组
2. 每次把未排序序列的第一个插入到已排序序列的合适位置



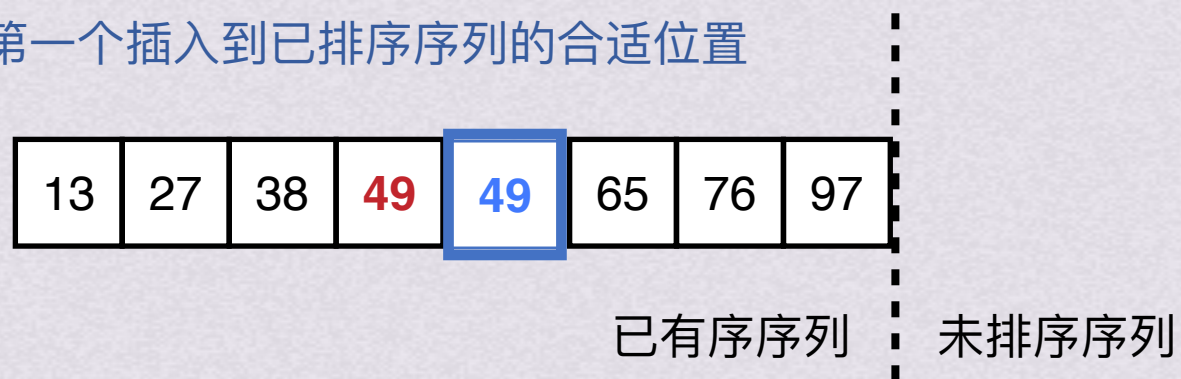


插入排序

直接插入排序

核心思想

1. 分割为已排序和未排序两组
2. 每次把未排序序列的第一个插入到已排序序列的合适位置





插入排序

直接插入排序

核心思想

1. 分割为已排序和未排序两组
2. 每次把未排序序列的第一个插入到已排序序列的合适位置

13	27	38	49	49	65	76	97
----	----	----	----	----	----	----	----

排序完成



插入排序

直接插入排序

```
void InsertSort(SqList &L)
{
    // 对顺序表 L 进行直接插入排序
    for(i=2; i<=L.length; ++i)
        if(L[i]<L[i-1]) //“<”, 需将 L[i] 插入有序子表
        {
            L[0]=L[i]; //将待插入的记录暂存到监视哨中
            L[i]=L[i-1]; //L[i-1]后移
            for(j=i-2; L[0]<L[j]; --j){ //从后向前寻找插入位置
                L[j+1]=L[j]; // 记录逐个后移, 直到找到插入位置
            }
            L[j+1]=L[0]; //将L[0]即原L[i], 插入正确位置
        } //if
    }
}
```




插入排序

直接插入排序

```
void InsertSort(SqList &L)
{
    // 对顺序表 L 进行直接插入排序
    for(i=2; i<=L.length; ++i)
        if(L[i]<L[i-1]) // "<", 需将 L[i] 插入有序子表
        {
            L[0]=L[i]; // 将待插入的记录暂存到监视哨中
            L[i]=L[i-1]; // L[i-1] 后移
            for(j=i-2; L[0]<L[j]; --j){ // 从后向前寻找插入位置
                L[j+1]=L[j]; // 记录逐个后移, 直到找到插入位置
            }
            L[j+1]=L[0]; // 将L[0]即原L[i], 插入正确位置
        }
    // if
}
```

1. 从 $i=2$ （待排序表的第2个元素）开始，如果该元素小于前一个元素，则需要插入

2. 每次插入之前都要先将当前元素复制到哨兵位置

3. 将当前元素前一个元素后移，覆盖当前元素

4. 在已有序表中寻找插入位置：

因为已判断过前一个元素和当前元素的大小，因此从 $j=i-2$ 即前前一个元素开始向前寻找插入位置，未找到时每一轮都让 j 指向的元素后移，给插入元素腾位置

5. 找到插入位置，将存放待插入元素的哨兵位置的元素复制到插入位置，完成插入

存储元素	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8



插入排序

直接插入排序

未排序状态

存储元素	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8

直接插入排序完成

存储元素	49	13	27	38	49	49	65	76	97
数组下标	0	1	2	3	4	5	6	7	8

排序前后均为先红49 后蓝49
直接插入排序是稳定的排序算法



插入排序

直接插入排序

未排序状态

存储元素	-	49	38	65	97	76	13	27	49
数组下标	0	1	2	3	4	5	6	7	8

直接插入排序完成

存储元素	49	13	27	38	49	49	65	76	97
数组下标	0	1	2	3	4	5	6	7	8

排序前后均为先红49 后蓝49
直接插入排序是稳定的排序算法

空间复杂度 $O(1)$

时间复杂度:

平均情况下 $O(n^2)$

最好情况下 $O(n)$

最坏情况下 $O(n^2)$



插入排序

折半插入排序



插入排序折半插入排序

```
void BInsertSort(SqList &L)
{
    // 对顺序表 L 进行折半插入排序
    for(i=2; i<=L.length; ++i)
    {
        L[0]=L[i]; //待插入的记录暂存在监视哨
        low=1; high=i-1; //置查找区间初值
        while(low<=high) //在L[low..high]中折半查找插入的位置
        {
            m=(low+high)/2; //折半
            if(L[0]<L[m]) high=m-1; //插入点在前一子表
            else low=m+1; //插入点在后一子表
        } //while
        for(j=i-1; j>=high+1; --j)
        {
            L[j+1]=L[j]; //记录后移
        }
        L[high+1]=L[0]; // 将L[0] 即原 L[i], 插入正确位置
    } //for
}
```



插入排序折半插入排序

```
void BInsertSort(SqList &L)
{
    // 对顺序表 L 进行折半插入排序
    for(i=2; i<=L.length; ++i)
    {
        L[0]=L[i]; //待插入的记录暂存在监视哨
        low=1; high=i-1; //置查找区间初值
        while(low<=high) //在L[low..high]中折半查找插入的位置
        {
            m=(low+high)/2; //折半
            if(L[0]<L[m]) high=m-1; //插入点在前一子表
            else low=m+1; //插入点在后一子表
        } //while
        for(j=i-1; j>=high+1; --j)
        {
            L[j+1]=L[j]; //记录后移
        }
        L[high+1]=L[0]; // 将L[0] 即原 L[i], 插入正确位置
    } //for
}
```

1. 从 $i=2$ （待排序表的第二个元素）开始进行折半插入排序
 $L[0]$ 存放待插入的记录

2. 在 low 到 $high$ 之间折半查找需要插入的位置

3. 向后移动需插入位置后所有元素，再将 $L[0]$ 插入回该位置

存储内容

-	27	49	13	38
---	----	----	----	----

数组下标

0 1 2 3 4



插入排序折半插入排序

```
void BInsertSort(SqList &L)
{
    // 对顺序表 L 进行折半插入排序
    for(i=2; i<=L.length; ++i)
    {
        L[0]=L[i]; //待插入的记录暂存在监视哨
        low=1; high=i-1; //置查找区间初值
        while(low<=high) //在L[low..high]中折半查找插入的位置
        {
            m=(low+high)/2; //折半
            if(L[0]<L[m]) high=m-1; //插入点在前一子表
            else low=m+1; //插入点在下一子表
        } //while
        for(j=i-1; j>=high+1; --j)
        {
            L[j+1]=L[j]; //记录后移
        }
        L[high+1]=L[0]; // 将L[0] 即原 L[i], 插入正确位置
    } //for
}
```

存储内容	38	13	27	38	49
数组下标	0	1	2	3	4

稳定算法

复杂度仍为 $O(n^2)$

只减少比较次数

移动次数未改变



插入排序

希尔排序
(缩小增量排序)



插入排序

希尔排序 (缩小增量排序)

基本思想

- 1.把待排序表相隔几个元素分割成几个子表
- 2.对每个子表分别进行直接插入排序
- 3.整个表中的元素已基本有序时，再对全体记录进行一次直接插入排序



插入排序

希尔排序

(缩小增量排序)

基本思想

- 1.把待排序表相隔几个元素分割成几个子表
- 2.对每个子表分别进行直接插入排序
- 3.整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序

49	38	65	97	76	13	27	49	55	04
----	----	----	----	----	----	----	----	----	----



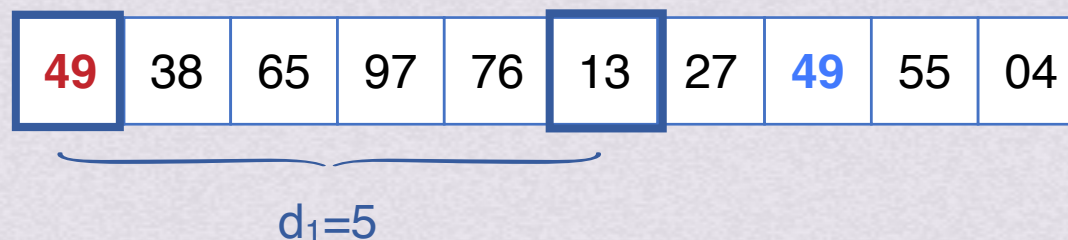
插入排序

希尔排序 (缩小增量排序)

基本思想

- 1.把待排序表相隔几个元素分割成几个子表
- 2.对每个子表分别进行直接插入排序
- 3.整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序





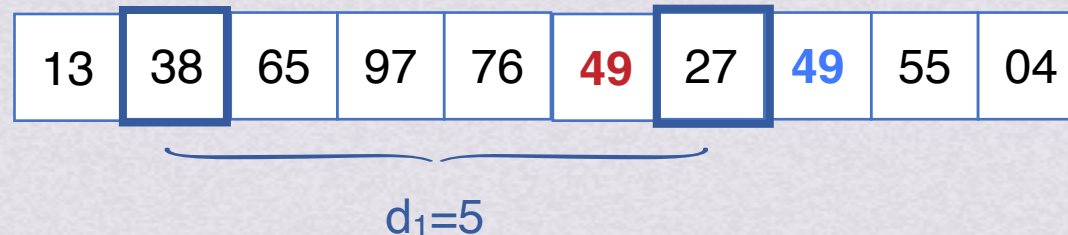
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$, $d_2=3$, 再直接插入排序





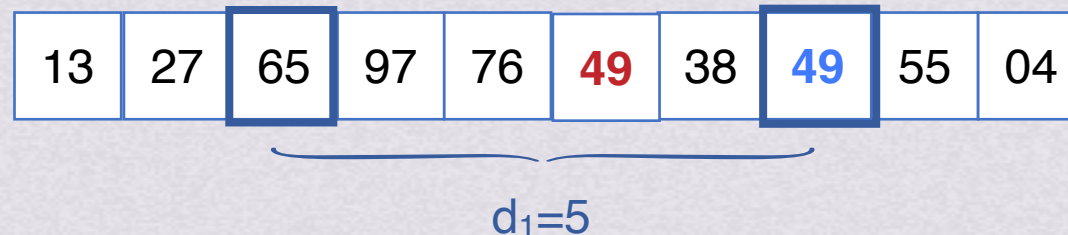
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$, $d_2=3$, 再直接插入排序





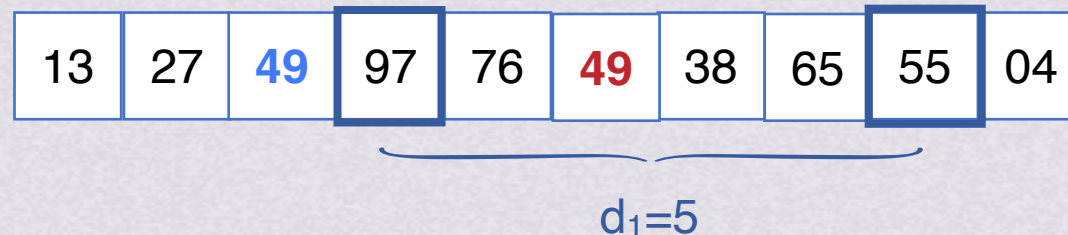
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序





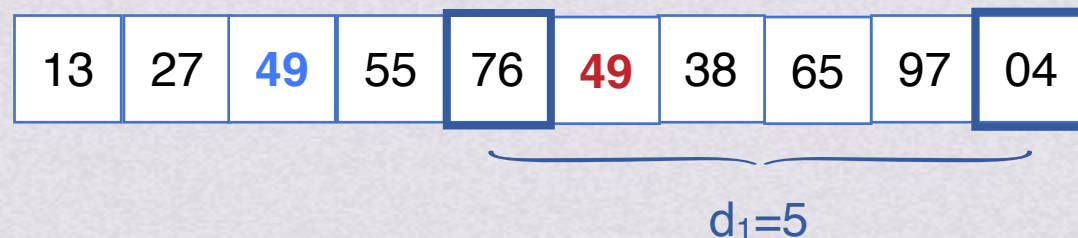
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序





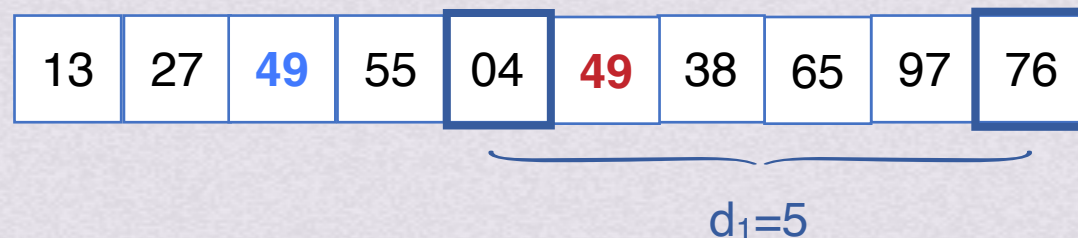
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序



第一趟排序完成
取第二趟排序的 $d_2=3$



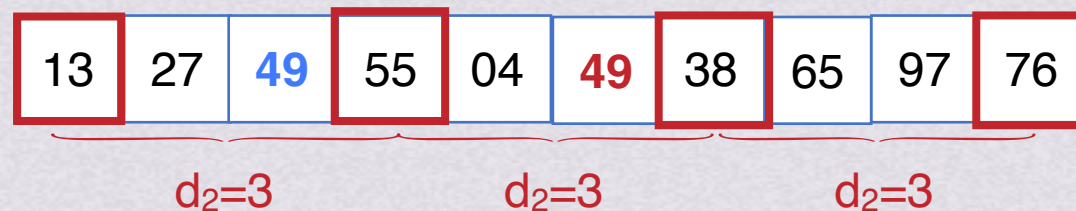
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序



第一趟排序完成
取第二趟排序的 $d_2=3$



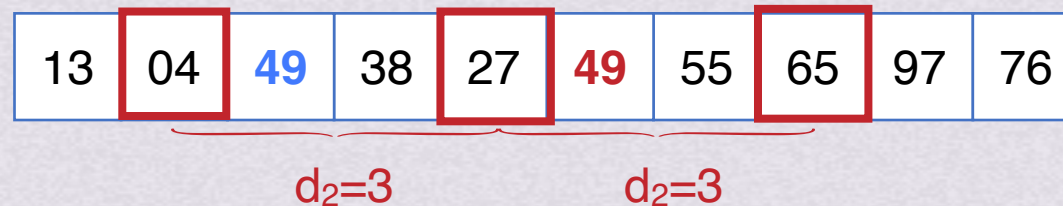
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序



第一趟排序完成
取第二趟排序的 $d_2=3$



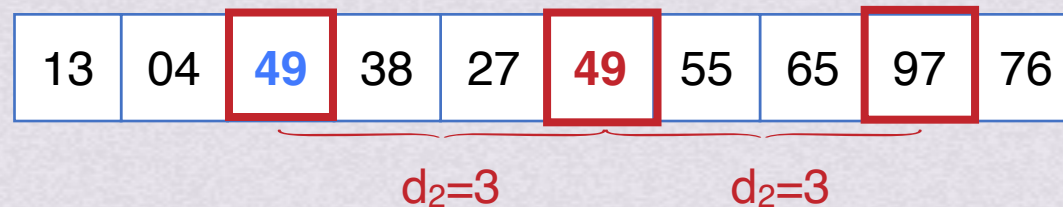
插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序



第一趟排序完成
取第二趟排序的 $d_2=3$



插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$, $d_2=3$, 再直接插入排序

13	04	49	38	27	49	55	65	97	76
----	----	----	----	----	----	----	----	----	----

已有序序列 | 未排序序列

第二趟排序完成
第三趟直接插入排序



插入排序

希尔排序 (缩小增量排序)

基本思想

1. 把待排序表相隔几个元素分割成几个子表
2. 对每个子表分别进行直接插入排序
3. 整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$, $d_2=3$, 再直接插入排序

04	13	27	38	49	49	55	65	97	76
----	----	----	----	----	----	----	----	----	----

已有序序列 | 未排序序列

第二趟排序完成
第三趟直接插入排序



插入排序

希尔排序 (缩小增量排序)

基本思想

- 1.把待排序表相隔几个元素分割成几个子表
- 2.对每个子表分别进行直接插入排序
- 3.整个表中的元素已基本有序时，在对全体记录进行一次直接插入排序

假设取 $d_1=5$ ， $d_2=3$ ，再直接插入排序

04	13	27	38	49	49	55	65	76	97
----	----	----	----	----	----	----	----	----	----

第三趟排序完成

希尔排序是不稳定算法！

时间复杂度涉及数学上尚未解决的难题，最坏情况下为 $O(n^2)$